



ESTRUTURAS DE DADOS E ALGORITMOS

Algoritmos Gananciosos

Aula 6 · a melhor escolha agora, sem voltar atrás

Prof. Doutor Luís Simões da Cunha
luiscunha@ismt.pt

Três perguntas para aquecer

1

Qual é a ideia central da programação dinâmica?

2

Com moedas de 1, 3 e 4, quantas moedas são precisas, no mínimo, para 6 cêntimos?

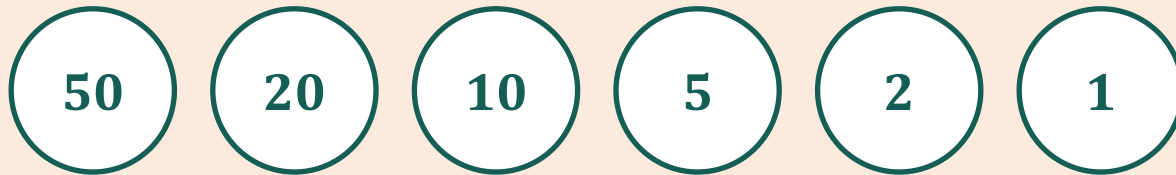
3

Qual a diferença entre a abordagem bottom-up e a top-down?

Responda sem olhar para os apontamentos: recordar é o que consolida a memória.

Está na caixa do café e tem de dar 48 cêntimos de troco. Que moedas escolhe?

Moedas disponíveis:



O que quase toda a gente faz



a maior moeda que ainda cabe, uma de cada vez

Sem pensar muito, escolheu sempre "o melhor agora". Isso é um algoritmo ganancioso.

Escolher bem a cada passo

1

A ideia

O troco, passo a passo.

2

As regras

Viável, localmente ótima,
irrevogável.

3

Funciona sempre?

Contraexemplos e como
provar.

4

Huffman

Comprimir dados com uma
árvore construída à
gananciosa.

Pergunta-guia: quando é que "o melhor agora" leva ao melhor no fim?

1

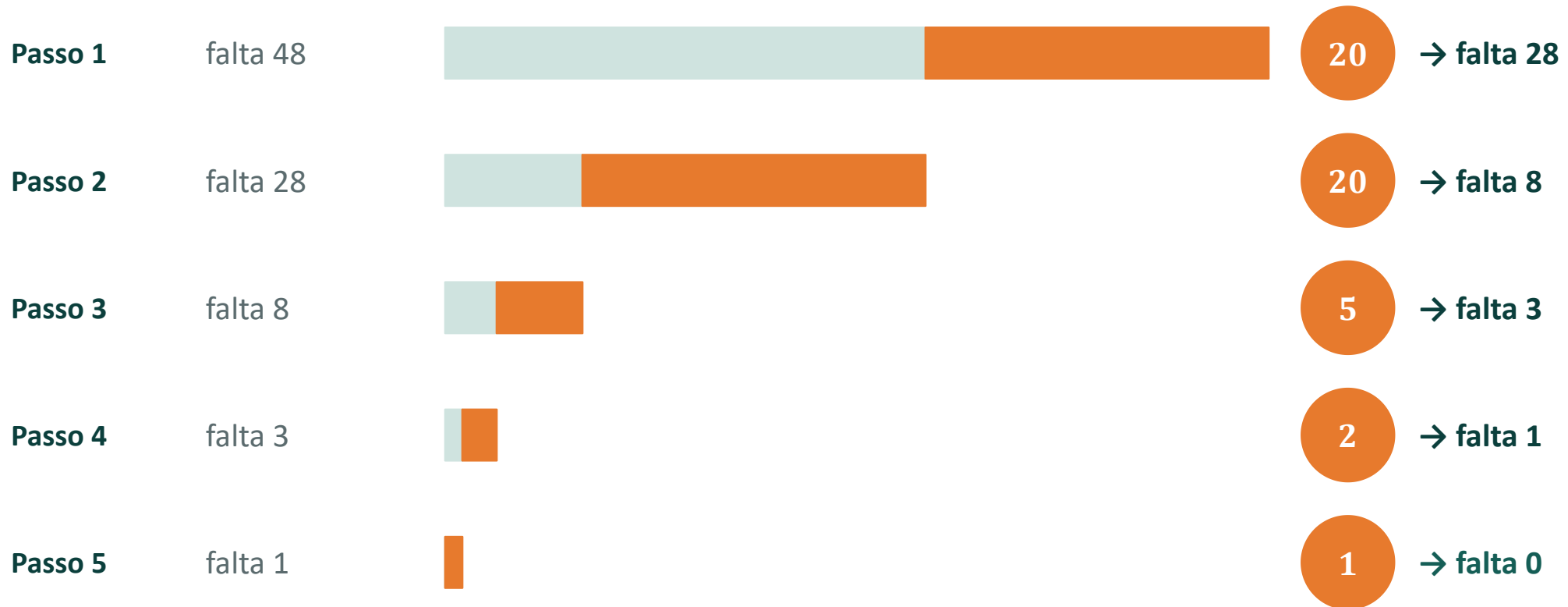
PARTE 1

A ideia gananciosa

O problema do troco, outra vez.

48 cêntimos, uma moeda de cada vez

Regra: dar sempre a maior moeda que ainda cabe no montante que falta.



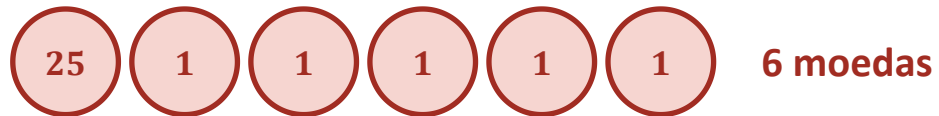
Resultado: $20 + 20 + 5 + 2 + 1 = 48$, com 5 moedas.

A solução gananciosa é sempre a melhor?

Com as moedas do euro: sim. Pode provar-se que a estratégia gananciosa dá sempre o número mínimo de moedas.

Moedas 25, 10, 1 · $n = 30$

Ganancioso



Ótimo



Moedas 1, 3, 4 · $n = 6$ (Aula 5)

Ganancioso



Ótimo



2

PARTE 2

As regras do jogo

O que faz de um algoritmo um algoritmo ganancioso.

Construir a solução peça a peça

Em cada passo, **acrescenta-se uma peça** à solução parcial, até a solução ficar completa. Só se aplica a **problemas de otimização**.

Em cada passo, a escolha tem de ser:

1 Viável

Respeita as restrições do problema.

No troco: a moeda não pode ultrapassar o que falta.

2 Localmente ótima

É a melhor entre as escolhas viáveis nesse passo.

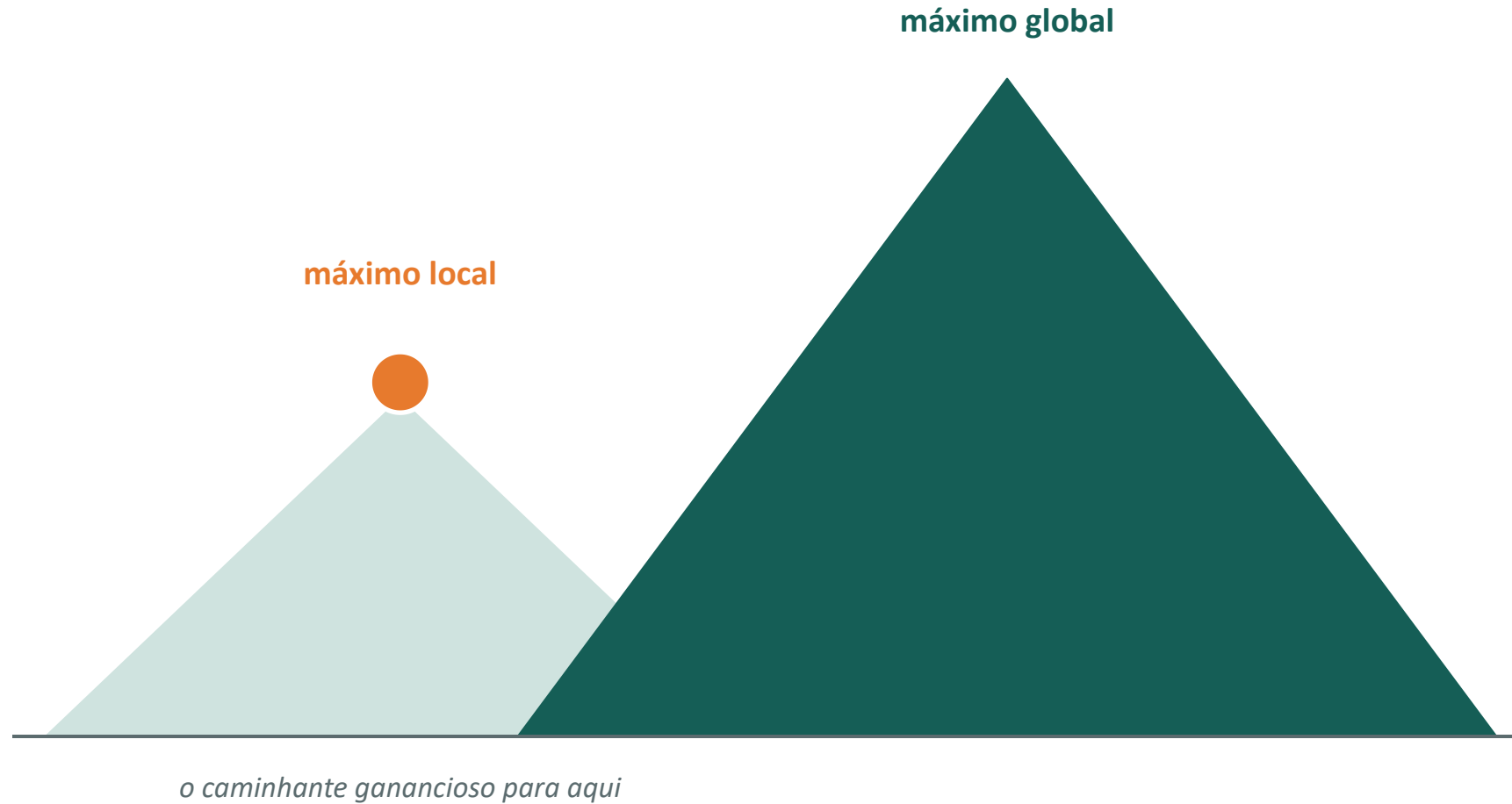
No troco: a maior moeda que cabe.

3 Irrevogável

Depois de feita, nunca mais é alterada.

No troco: a moeda dada não volta para a caixa.

O melhor agora nem sempre é o melhor no fim



A aposta

O algoritmo ganancioso aposta que uma sequência de escolhas localmente ótimas dá uma solução globalmente ótima.

Às vezes ganha. Às vezes não.

Quatro formas de atacar um problema

Estratégia	Ideia central	Garante o ótimo?	Exemplo
Força bruta (Aula 3)	Experimentar tudo	Sim	Caixeiro-viajante
Divisão e conquista (Aula 4)	Partir em subproblemas independentes	Sim	Merge Sort
Programação dinâmica (Aula 5)	Guardar subproblemas repetidos	Sim	Troco (qualquer moeda)
Gananciosa (hoje)	O melhor agora, sem voltar atrás	Só em alguns problemas	Troco (euro), Huffman

A grande vantagem da gananciosa: é simples e rápida — um só caminho, sem tabelas nem recuos.

3

PARTE 3

Funciona sempre?

Fácil de conceber, difícil de provar.

Onde a estratégia gananciosa ganha — e onde perde

Dá sempre o ótimo

- Troco com as moedas do euro
- Codificação de Huffman (hoje)
- Árvores de cobertura mínima (Prim, Kruskal)
- Caminhos mais curtos (Dijkstra)

Não garante o ótimo

- Troco com moedas quaisquer (25, 10, 1)
- Caixeiro-viajante: ir sempre à cidade mais próxima
- Mochila com objetos indivisíveis

→ *mas pode dar uma boa aproximação, muito depressa*

Regra geral: descobrir uma estratégia gananciosa é fácil, sobretudo depois de experimentar alguns exemplos. O difícil é provar que dá sempre o ótimo.

Três caminhos para provar que é ótimo

1

Indução matemática

Mostrar que a solução parcial de cada passo pode sempre ser completada até uma solução ótima.

2

"Fica sempre à frente"

Mostrar que, em cada passo, o ganancioso avança pelo menos tanto como qualquer outro algoritmo.

3

Pelo resultado final

Mostrar que o resultado é ótimo analisando o resultado, e não o modo como foi obtido.

E para provar que NÃO é ótimo? Basta um contraexemplo — como as moedas 25, 10, 1 com $n = 30$.

4

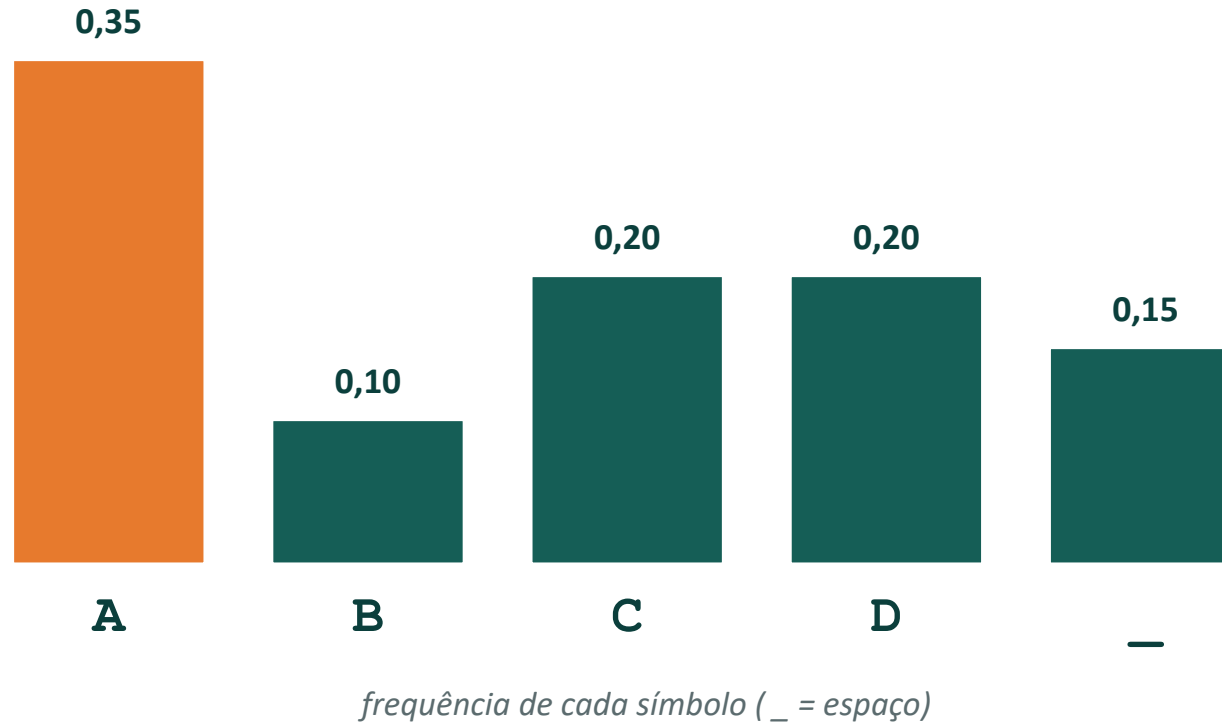
PARTE 4

Codificação de Huffman

Comprimir dados, à gananciosa.

Símbolos frequentes, códigos curtos

Texto com 5 símbolos. Algumas letras aparecem muito mais do que outras:



Código fixo

3 bits por símbolo

A = 000, B = 001, C = 010... todos iguais, frequentes ou raros.

Código de Huffman

comprimento variável

Como no código Morse: a letra mais usada (E) é um só ponto.

Juntar sempre as duas árvores mais leves

1

Começar

Uma árvore de um só nó por símbolo, com peso = frequência.

2

Repetir

Juntar as duas árvores de menor peso numa nova árvore. O peso da nova árvore é a soma dos dois.

3

Parar

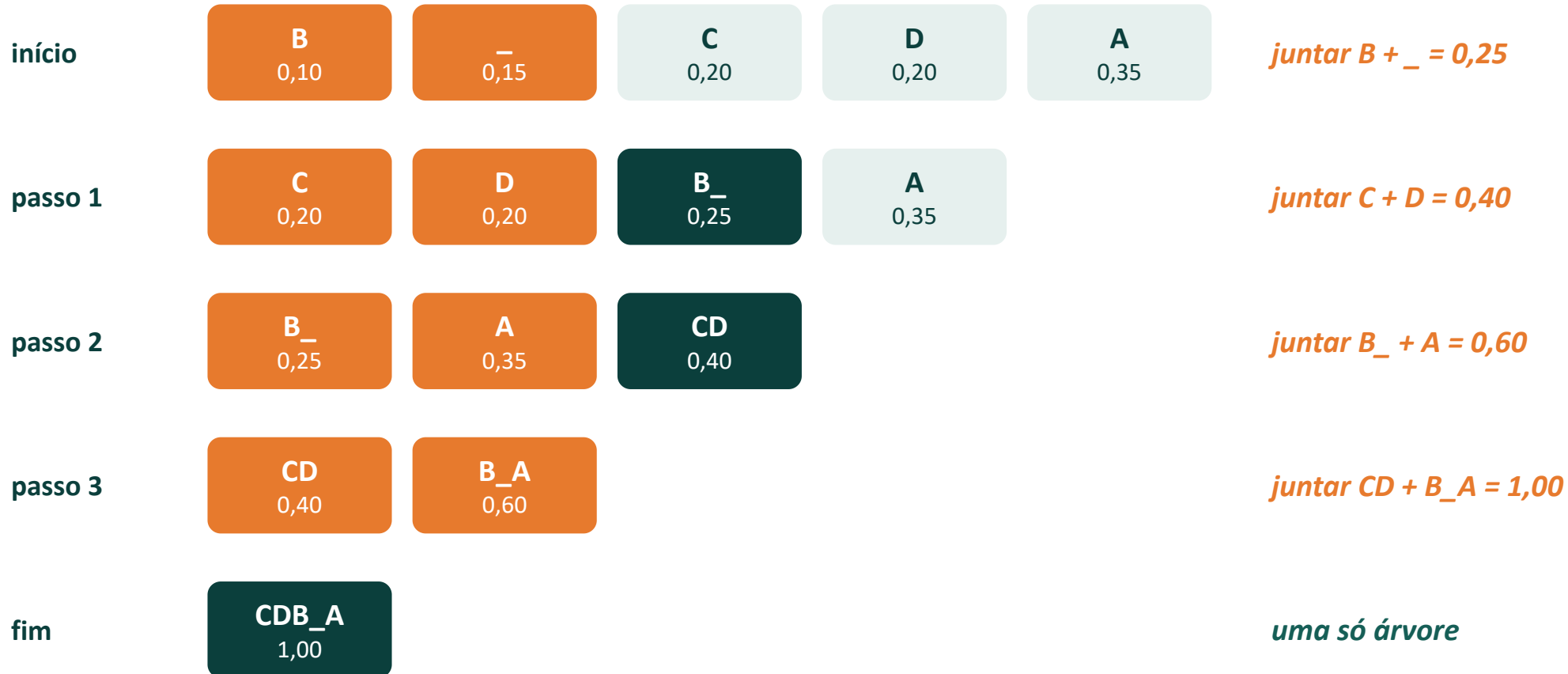
Quando só restar uma árvore. Ramos à esquerda = 0, à direita = 1.

Onde está a ganância?

Em cada passo escolhemos as duas árvores mais leves — a melhor escolha local — e nunca desfazemos uma junção.

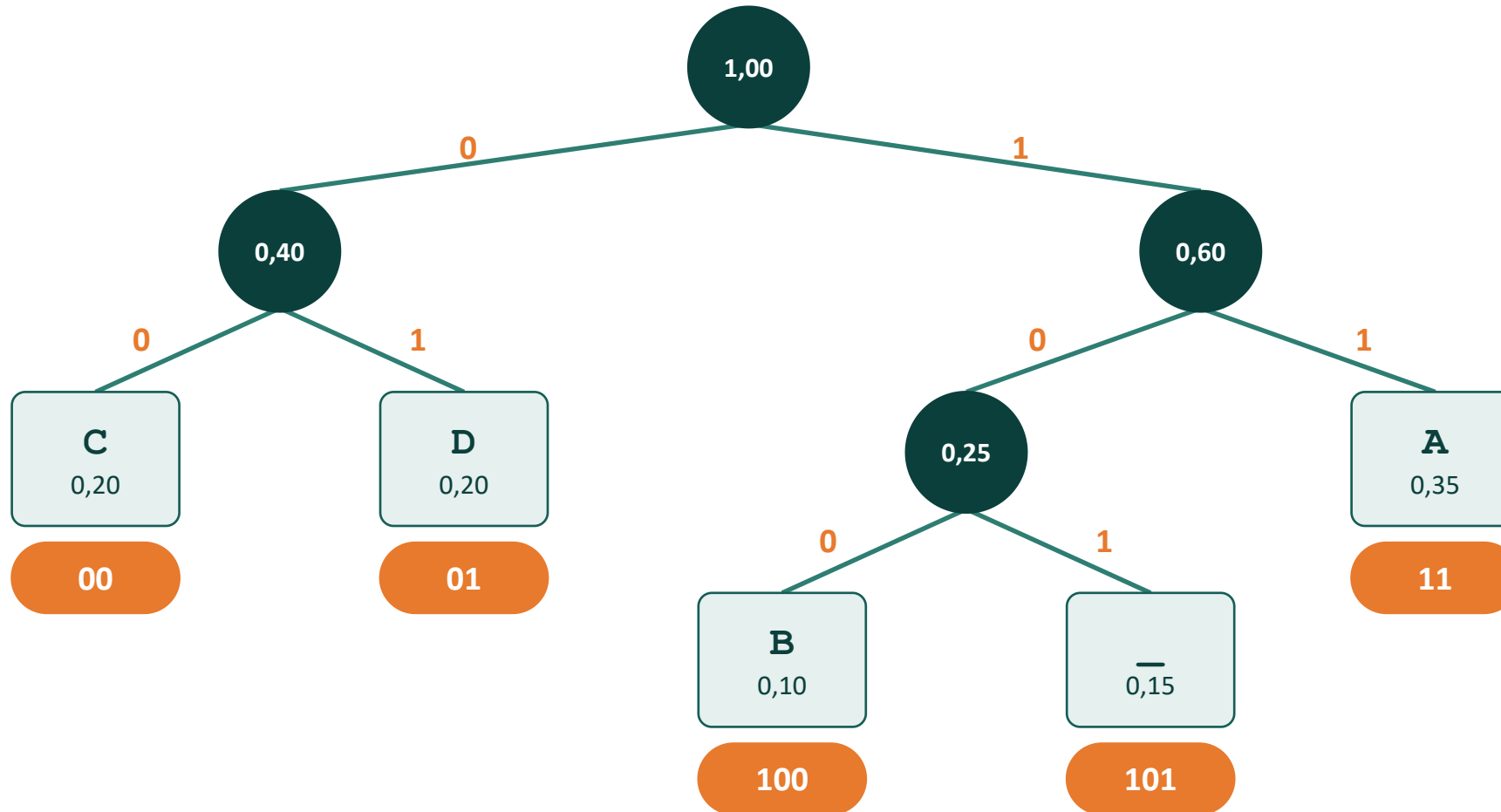
Símbolos raros ficam mais fundo na árvore → códigos mais compridos.

Construir a árvore passo a passo



Laranja = as duas mais leves (vão ser juntadas). Verde escuro = árvore já construída.

Ler os códigos pelos ramos



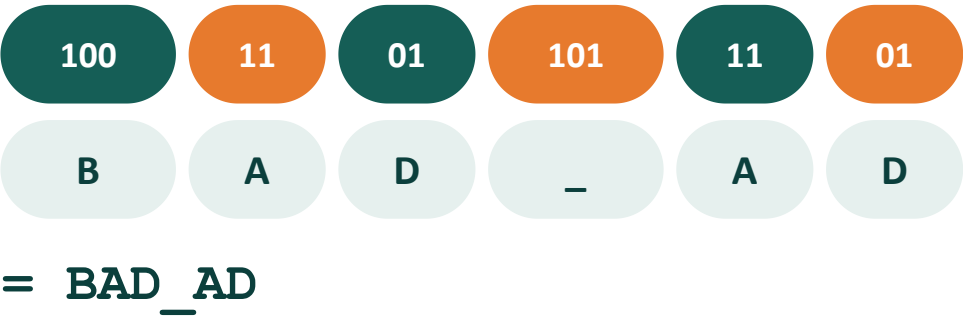
Codificar e decodificar

Símbolo	A	B	C	D	_
Frequência	0,35	0,10	0,20	0,20	0,15
Código	11	100	00	01	101

Codificar "DAD"



Descodificar "10011011011101"



Porque não há ambiguidade? Nenhum código é o início de outro: todos os símbolos estão nas folhas da árvore.

QUANTO SE POUPA?

Número médio de bits por símbolo

$$2 \cdot 0,35 + 3 \cdot 0,10 + 2 \cdot 0,20 + 2 \cdot 0,20 + 3 \cdot 0,15$$

Código fixo

3

bits por símbolo

Huffman

2,25

bits por símbolo

Poupança

25 %

menos espaço

Huffman está na base de formatos que usa todos os dias, como ZIP, PNG e JPEG.

Agora é consigo



Troco ganancioso



Implemente em Java o algoritmo ganancioso para o problema do troco.

Teste com o euro ($48 \rightarrow 20+20+5+2+1$) e com 25, 10, 1 para $n = 30$. O que acontece?



Huffman



Implemente em Java a codificação de Huffman com o algoritmo ganancioso.

Pista: uma PriorityQueue dá-lhe sempre as duas árvores mais leves. Teste com o exemplo ($A = 11, B = 100\dots$).



Mini-projeto 1



Implemente em Java uma solução gananciosa para o problema do mini-projeto 1.

Compare com a solução de programação dinâmica: dão o mesmo resultado?

Três ideias a reter

1

O melhor agora

Cada escolha é viável, localmente ótima e irrevogável.

2

Nem sempre é ótimo

Funciona para o euro e para Huffman; falha com moedas 25, 10, 1. Um contraexemplo basta.

3

Huffman

Juntar sempre as duas árvores mais leves: códigos curtos para símbolos frequentes.

Programação dinâmica pondera todas as opções; a gananciosa aposta numa só.

Bibliografia

1

Levitin, A. (2011). *Introduction to the Design and Analysis of Algorithms* (3.ª ed.). Pearson. Capítulo 9: Greedy Technique.

Dúvidas? Prof. Doutor Luís Simões da Cunha · luiscunha@ismt.pt